

LAB 14

NAME: Hajra Sarwar

REG NO: 2023-BSE-022

CLASS: BSE-5A

SUBJECT: CC

LAB TITLE: Terraform + Ansible: Dynamic Inventory, Roles & Automated Nginx/PHP & Docker Deployment

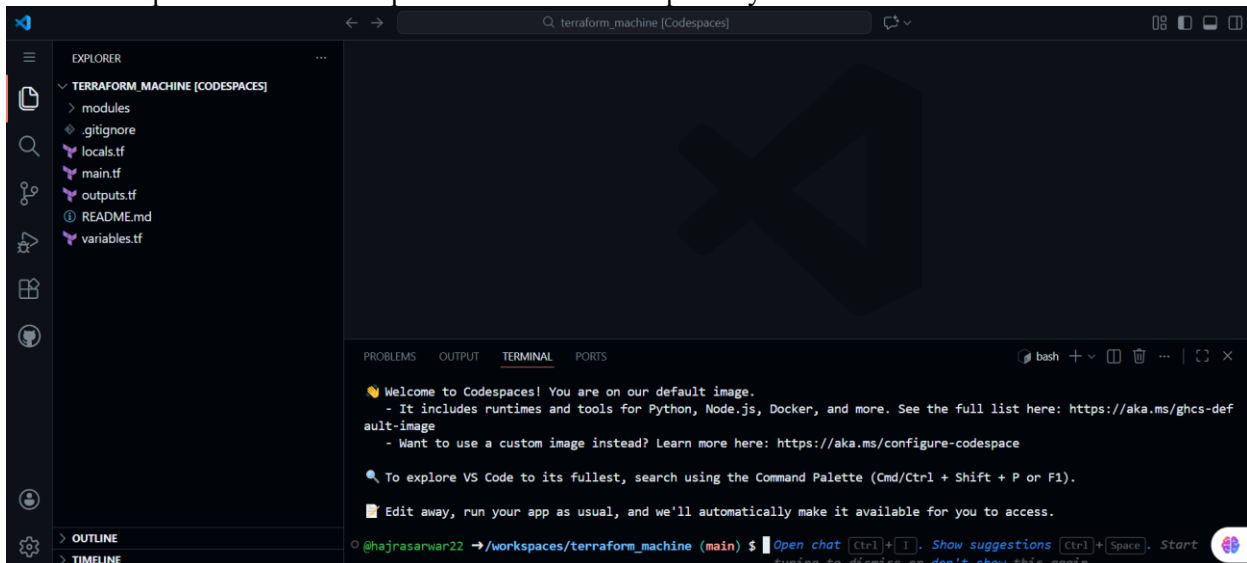
TASK:

Task 0 – Lab Setup (Codespace & GH CLI)

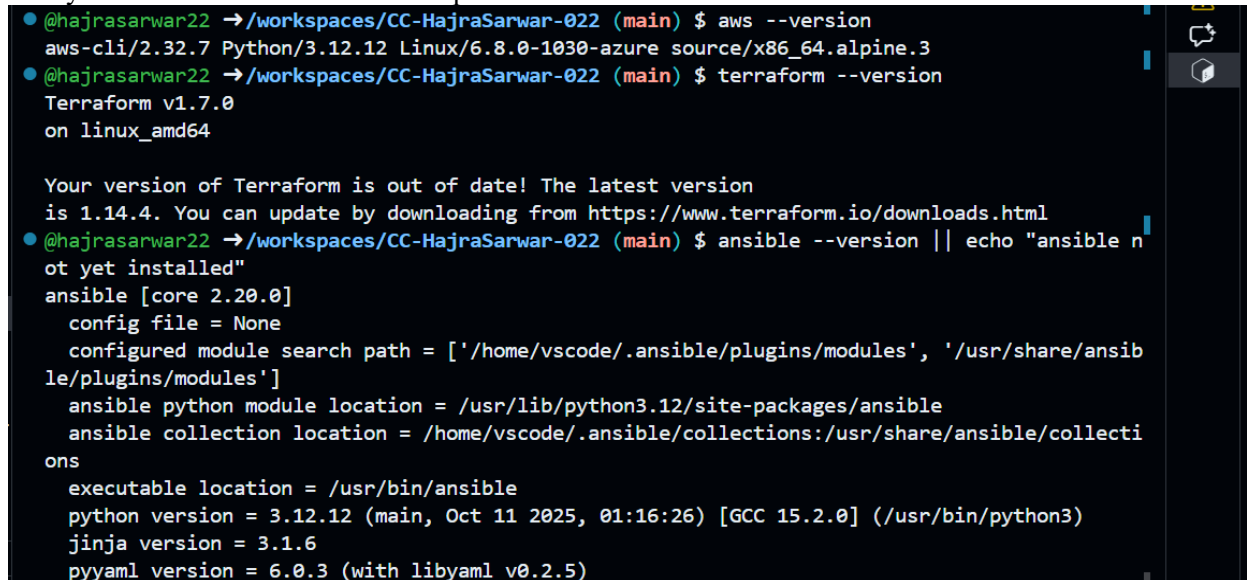
This task prepares the required development environment using GitHub Codespaces and ensures that all tools and credentials are correctly configured before starting the lab.

Steps

1. Fork the **terraform_machine** repository to your GitHub account.
2. Create and open a GitHub Codespace from the forked repository.



3. Verify installed tools inside the Codespace terminal:



4. Verify AWS CLI authentication and permissions:

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022 (main) $ aws sts get-caller-identity
{
  "UserId": "AIDA5WPGPNRI7ANGZM5NF",
  "Account": "941618064465",
  "Arn": "arn:aws:iam::941618064465:user/Admin"
}
```

Task 1 – Generate SSH Key and Initial Terraform Apply

This task initializes the Terraform project, generates SSH keys for EC2 access, and provisions two EC2 instances using the existing Terraform configuration.

Steps

1. Check existing SSH keys and generate a new SSH key pair if required:

Before key generation:

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ls ~/.ssh
ls: cannot access '/home/vscode/.ssh': No such file or directory
```

Key generation:

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ssh-keygen -t ed25519 -f
~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Created directory '/home/vscode/.ssh'.
Your identification has been saved in /home/vscode/.ssh/id_ed25519
Your public key has been saved in /home/vscode/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:f+2ojaUiNqXjngMrLFGL6jan/Yq3h8fJgAnK89aqe6A vscode@codespaces-079caa
The key's randomart image is:
+---[ED25519 256]---+
|
|..
|+oo. S
|+*... ..
|oo+ *o.o . o .
|Eo=B.BB.. * o
|oBX*B=+= .+o .
+----[SHA256]-----+
```

After key generation:

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ls -la ~/.ssh
total 20
drwx----- 2 vscode vscode 4096 Jan 28 16:17 .
drwxr-xr-x 1 vscode vscode 4096 Jan 28 16:17 ..
-rw----- 1 vscode vscode 419 Jan 28 16:17 id_ed25519
-rw-r--r-- 1 vscode vscode 106 Jan 28 16:17 id_ed25519.pub
```

2. Navigate to the project directory and create terraform.tfvars.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ touch terraform.tfvars
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ls -la terraform.tfvars
-rw-rw-rw- 1 vscode vscode 0 Jan 28 16:37 terraform.tfvars
```

3. Add required Terraform variables including VPC CIDR, subnet CIDR, instance type, and SSH keys.

```
lab_14 > Task > terraform.tfvars
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 availability_zone = "me-central-1a"
4 env_prefix = "dev"
5 instance_type = "t3.micro"
6 public_key = "~/.ssh/id_ed25519.pub"
7 private_key = "~/.ssh/id_ed25519"
```

4. Initialize Terraform using `terraform init`.

```
@hajrasarwar22 →.../lab_14/Task/modules/webserver (main) $ terraform init
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

5. Apply Terraform to create two EC2 instances:

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform apply -auto-ap
prove
Apply complete! Resources: 10 added, 0 changed, 0 destroyed.

Outputs:

webserver_public_ips = [
  "3.28.74.36",
  "158.252.33.79",
]
```

6. Display Terraform outputs to retrieve EC2 public IPs.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform output
webserver_public_ips = [
  "3.28.74.36",
  "158.252.33.79",
]
```

Task 2 – Static Ansible Inventory with Two EC2 Instances

This task installs Ansible, creates a static inventory file, and verifies connectivity between Ansible and EC2 instances.

Steps

1. Install Ansible Core using `pipx` and verify installation.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible --version
ansible [core 2.20.0]
  config file = None
  configured module search path = ['/home/vscode/.ansible/plugins/modules', '/usr/share/ansib
le/plugins/modules']
  ansible python module location = /usr/lib/python3.12/site-packages/ansible
  ansible collection location = /home/vscode/.ansible/collections:/usr/share/ansible/collecti
ons
  executable location = /usr/bin/ansible
  python version = 3.12.12 (main, Oct 11 2025, 01:16:26) [GCC 15.2.0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
```

2. Retrieve EC2 public IPs using Terraform output.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform output
webserver_public_ips = [
  "3.28.74.36",
  "158.252.33.79",
]
```

3. Create an Ansible inventory file named `hosts`.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ touch hosts
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ls -la hosts
-rw-rw-rw- 1 vscode vscode 0 Jan 28 17:51 hosts
```

4. Add both EC2 public IPs along with SSH user and private key details.

```

@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cat hosts
3.28.74.36      ansible_user=ec2-user      ansible_ssh_private_key_file=~/.ssh/id_ed25519
158.252.33.79  ansible_user=ec2-user      ansible_ssh_private_key_file=~/.ssh/id_ed25519

```

5. Test connectivity using Ansible ping module.

```

@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cd /workspaces/CC-HajraSarwar-022/lab_14/Task && chmod 755 . && ansible all -i hosts -m ping
158.252.33.79 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.74.36' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}

```

6. Disable strict host key checking if required and re-test connectivity.

```

@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible_ssh_common_args='-o StrictHostKeyChecking=no'
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible all -i hosts -m ping
.html for more information.
158.252.33.79 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.74.36' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}

```

Task 3 – Scale to Three Instances & Group-Based Inventory

This task scales EC2 instances to three using Terraform and restructures the Ansible inventory using host groups.

Steps

1. Update the Terraform webserver module to use `count = 3`.

```

lab_14 > Task > main.tf
23 module "myapp-webserver" {
29   my_ip = local.my_ip
30   vpc_id = aws_vpc.myapp_vpc.id
31   subnet_id = module.myapp-subnet.subnet.id
32
33   # Loop count
34   count = 3
35   # Use count.index to differentiate instances
36   instance_suffix = count.index
37

```

2. Apply Terraform to provision three EC2 instances.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform apply -auto-approve
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

webserver_public_ips = [
  "3.28.74.36",
  "158.252.33.79",
  "40.172.100.131",
]
```

3. Verify outputs showing three public IPs.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform output
webserver_public_ips = [
  "3.28.74.36",
  "158.252.33.79",
  "40.172.100.131",
]
```

4. Rewrite the Ansible inventory using group definitions [ec2] and [droplet].

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cat hosts
[ec2]
3.28.74.36
158.252.33.79

[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'

[droplet]
40.172.100.131

[droplet:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

5. Test connectivity for individual groups, single hosts, and all instances.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible ec2 -i hosts -m ping
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '158.252.33.79' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
158.252.33.79 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
```

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible 3.28.74.36 -i hosts -m ping
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
```

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible droplet -i hosts -m ping
40.172.100.131 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}

@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible all -i hosts -m ping
158.252.33.79 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '40.172.100.131' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
40.172.100.131 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.74.36' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
```

Task 4 – Global Ansible Configuration & First Nginx Playbook

This task configures a global Ansible configuration file and deploys Nginx using a basic playbook.

Steps

1. Create a global Ansible configuration file at `~/.ansible.cfg`.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS vim - Task
[default]
host_key_checking = False
interpreter_python = /usr/bin/python3
~
~
~
```

2. Disable strict host key checking globally.

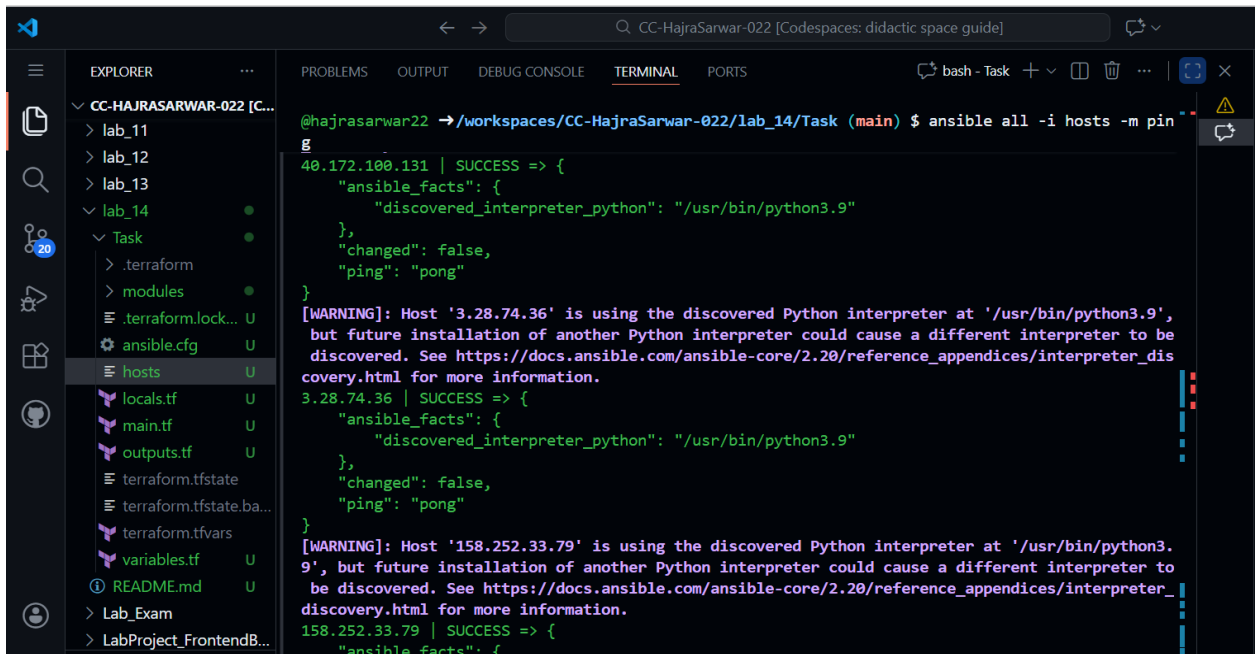
```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cd /workspaces/CC-HajraSarwar-022/lab_14/Task && cat hosts
[ec2]
3.28.74.36
158.252.33.79

[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519

[droplet]
40.172.100.131

[droplet:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

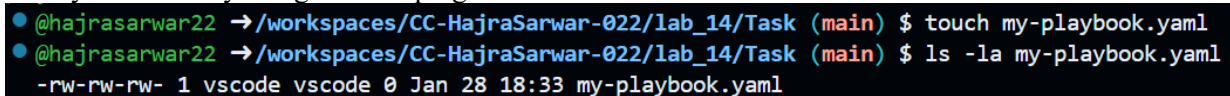
3. Remove SSH common arguments from inventory file.



The screenshot shows a VS Code window with a terminal. The Explorer sidebar on the left shows a project structure for 'CC-HAJRASARWAR-022'. The terminal displays the output of the command 'ansible all -i hosts -m ping'. It shows successful ping results for hosts 40.172.100.131 and 3.28.74.36, and a warning for host 158.252.33.79 about the Python interpreter. The output includes details about discovered facts like 'discovered_interpreter_python' and 'ping'.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible all -i hosts -m ping
40.172.100.131 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.74.36' is using the discovered Python interpreter at '/usr/bin/python3.9',
but future installation of another Python interpreter could cause a different interpreter to be
discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_dis
covery.html for more information.
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '158.252.33.79' is using the discovered Python interpreter at '/usr/bin/python3.
9', but future installation of another Python interpreter could cause a different interpreter to
be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter
discovery.html for more information.
158.252.33.79 | SUCCESS => {
  "ansible_facts": {
```

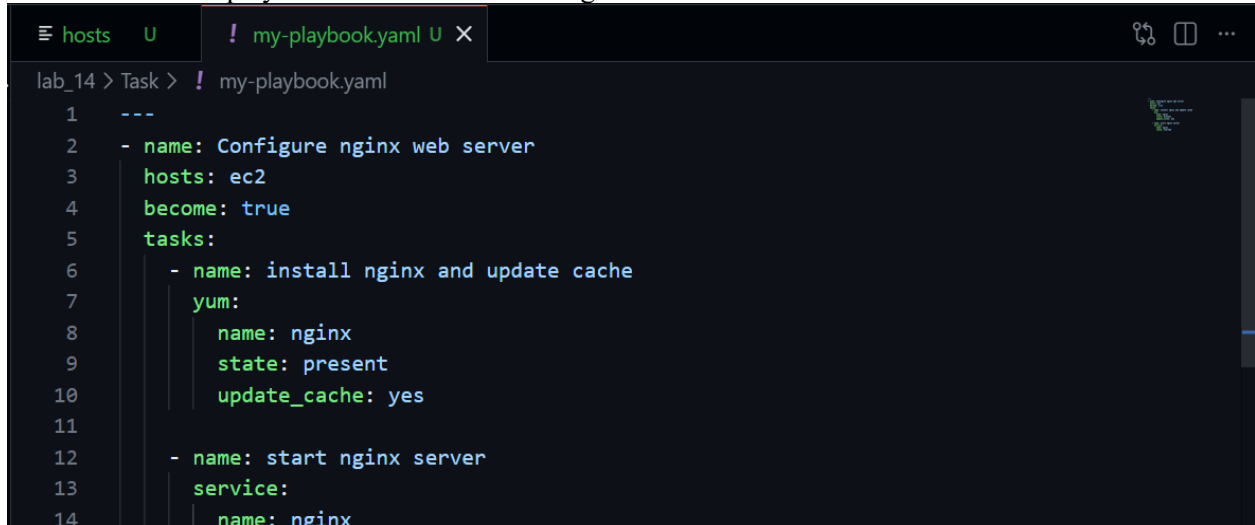
4. Verify connectivity using Ansible ping.



The screenshot shows a terminal window with two commands and their outputs. The first command creates a file 'my-playbook.yaml' using 'touch'. The second command lists the file with 'ls -la', showing its permissions, owner, and creation time.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ touch my-playbook.yaml
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ls -la my-playbook.yaml
-rw-rw-rw- 1 vscode vscode 0 Jan 28 18:33 my-playbook.yaml
```

5. Create an Ansible playbook to install and start Nginx.



The screenshot shows a VS Code editor with a file named 'my-playbook.yaml'. The file contains an Ansible playbook with two tasks: 'install nginx and update cache' and 'start nginx server'. The first task uses the 'yum' module to install 'nginx' and update the cache. The second task uses the 'service' module to start the 'nginx' service.

```
lab_14 > Task > ! my-playbook.yaml
1 ---
2 - name: Configure nginx web server
3   hosts: ec2
4   become: true
5   tasks:
6     - name: install nginx and update cache
7       yum:
8         name: nginx
9         state: present
10        update_cache: yes
11
12    - name: start nginx server
13      service:
14        name: nginx
```

6. Execute the playbook on the [ec2] group.



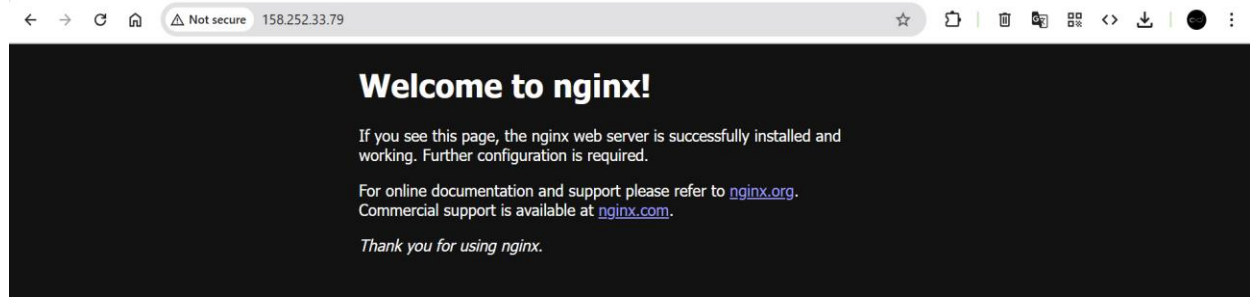
The screenshot shows a terminal window with the output of the command 'ansible-playbook -i hosts my-playbook.yaml'. It displays the execution of two tasks: 'install nginx and update cache' and 'start nginx server'. The output shows that both tasks were successful on all hosts. A 'PLAY RECAP' section at the bottom summarizes the results for hosts 158.252.33.79 and 3.28.74.36, showing that all tasks were successful (ok=3) and no changes were made (changed=2).

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i hosts my-playbook.yaml
TASK [install nginx and update cache] *****
changed: [3.28.74.36]
changed: [158.252.33.79]

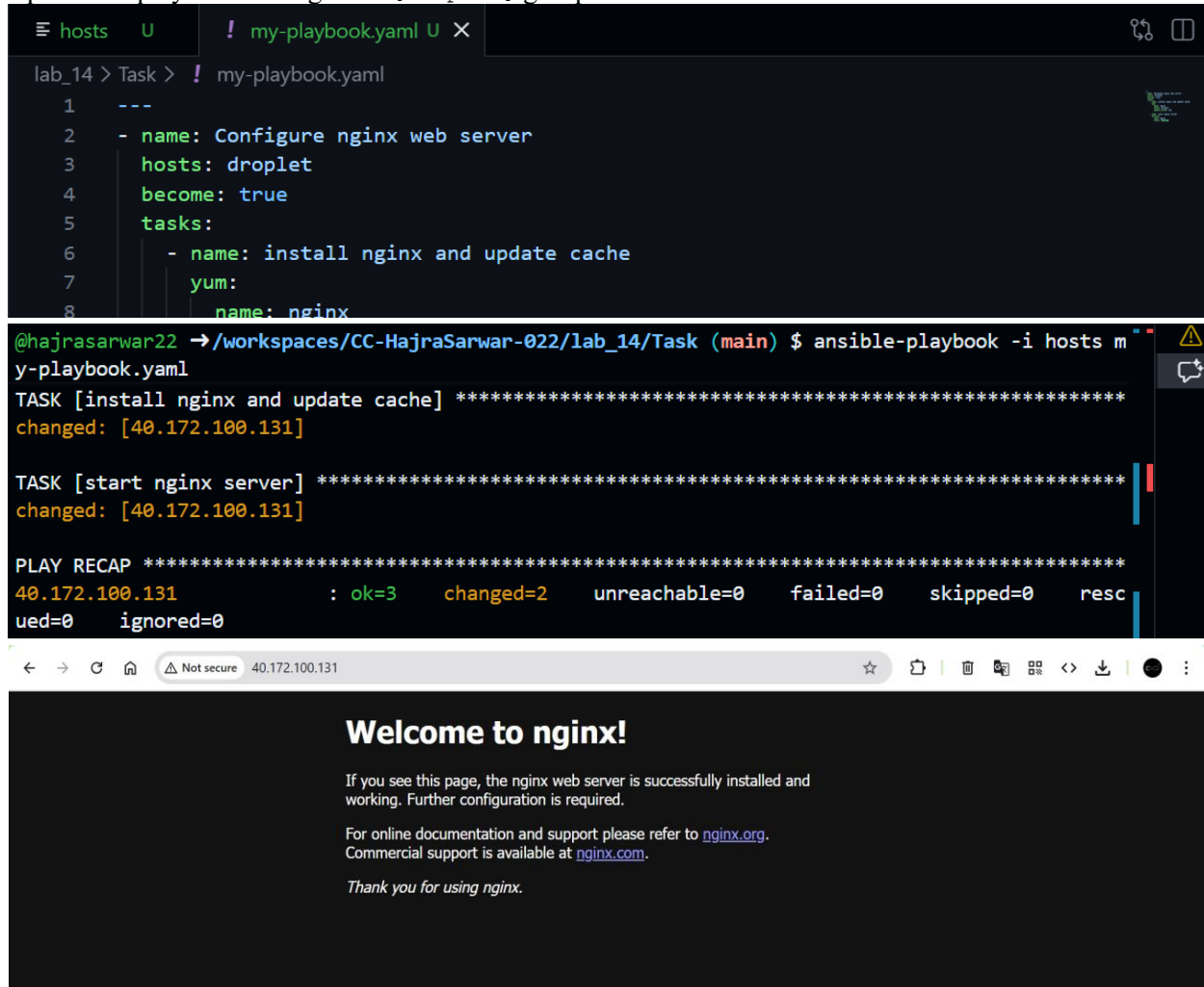
TASK [start nginx server] *****
changed: [158.252.33.79]
changed: [3.28.74.36]

PLAY RECAP *****
158.252.33.79 : ok=3 changed=2 unreachable=0 failed=0 skipped=0 resc
ued=0 ignored=0
3.28.74.36 : ok=3 changed=2 unreachable=0 failed=0 skipped=0 resc
ued=0 ignored=0
```

7. Verify Nginx deployment using a web browser.



8. Update the playbook to target the `[droplet]` group and re-run.



Task 5 – Single Nginx Target Group & HTTPS Prerequisites

This task configures a single Nginx server and prepares HTTPS prerequisites.

Steps

1. Create a project-level `ansible.cfg` file in the repository root.



2. Update Terraform to reduce EC2 count to one instance.


```
lab_14 > Task > main.tf
23 module "myapp-webserver" {
30     vpc_id = aws_vpc.myapp_vpc.id
31     subnet_id = module.myapp-subnet.subnet.id
32
33     # Loop count
34     count = 1
35     # Use count.index to differentiate instances
36     instance_suffix = count.index
37 }
```

3. Apply Terraform changes and retrieve the public IP.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform apply -auto-approve
Apply complete! Resources: 0 added, 0 changed, 6 destroyed.
```

4. Update the Ansible inventory to define a [nginx] group.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform output
webserver_public_ips = [
    "3.28.74.36",
]
```

5. Update the Nginx playbook to install OpenSSL and enable services.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cd /workspaces/CC-HajraSarwar-022/lab_14/Task && cat hosts
[nginx]
3.28.74.36

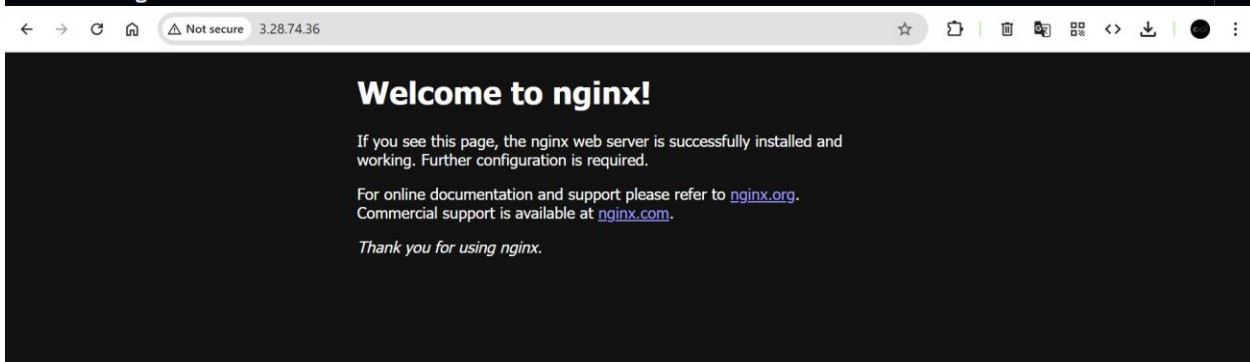
[nginx:vars]
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_user=ec2-user
```

6. Run the playbook and verify Nginx using a browser.

```
lab_14 > Task > my-playbook.yaml
1 ---
2 - name: Configure nginx web server
3   hosts: nginx
4   become: true
5   tasks:
6     - name: install nginx and update cache
7       yum:
8         name: nginx
```

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i hosts my-playbook.yaml
changed: [3.28.74.36]
```

```
PLAY RECAP *****
3.28.74.36 : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```



Task 6 – Ansible-Managed SSL Certificates

This task automates the creation of self-signed SSL certificates using Ansible and EC2 metadata.

Steps

1. Extend the existing [playbook](#) to include SSL certificate tasks.

```
lab_14 > Task > ! my-playbook.yaml U X ansible.cfg U main.tf U
2 - name: Configure nginx web server
22
23 - name: Configure SSL certificates
24   hosts: nginx
25   become: true
26   tasks:
27     - name: Create SSL private directory
28       file:
29         path: /etc/ssl/private
30         state: directory
31         mode: '0700'
32
33     - name: Create SSL certs directory
34       file:
```

2. Create required directories for SSL keys and certificates.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i hosts m
y-playbook.yaml
TASK [Get IMDSv2 token] *****
ok: [3.28.74.36]

TASK [Get current public IP] *****
ok: [3.28.74.36]

TASK [Show current public IP] *****
ok: [3.28.74.36] => {
  "msg": "Public IP: 3.28.74.36"
}

TASK [Generate self-signed SSL certificate] *****
changed: [3.28.74.36]

PLAY RECAP *****
3.28.74.36 : ok=11 changed=3 unreachable=0 failed=0 skipped=0 resc
ued=0 ignored=0
```

3. Retrieve EC2 public IP dynamically using IMDSv2.
4. Generate self-signed SSL certificates using OpenSSL.
5. Verify SSL certificate and key files via SSH.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ssh ec2-user@3.28.74.36 -i
~/.ssh/id_ed25519
-----BEGIN CERTIFICATE-----
MIIDOTCCAIGgAwIBAgIUdHsr1NBx7xjvonqcIFzoyCz+nIowDQYJKoZIhvcNAQEL
BQAwFTETMBEGA1UEAwKM4yOC43NC4zNjAeFw0yNjAxMjg0OTA3MzBaFw0yNzAx
Mjg0OTA3MzBaMBUxEzARBgNVBAMMCjMuMjguNzQuMzYwggEiMA0GCSqGSIb3DQEB
AQUAA4IBDwAwggEKAoIBAQCYS+o/3Yf87MvBGbnA8pN/FuyxitkkiCD6bhY5WVOA
AwVvKrPH28TBs3PDD2aREM7yWi5hqZMUfdWxkT9GDznEK9dBedHCGmXiGtvoHGH
RNQwlfCWfRdBwz9gHdvQw5W0uWpyobK+hkB8/mfElGiTr1hrX/cYBsyJrVA2Q2Ts
Ohv+9A3QlTrHYq4MPOAeYsOAK1kzWlccxao+MTK+Hh1bXtRhH+LerYpVn3pDOU94
[ec2-user@ip-10-0-10-145 ~]$ sudo cat /etc/ssl/private/selfsigned.key
-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBAgEAAoIBAQCYS+o/3Yf87MvB
GbnA8pN/FuyxitkkiCD6bhY5WVOAAwVvKrPH28TBs3PDD2aREM7yWi5hqZMUfdW
xkT9GDznEK9dBedHCGmXiGtvoHGHNRNqWlfCWfRdBwz9gHdvQw5W0uWpyobK+hkB8
/mfElGiTr1hrX/cYBsyJrVA2Q2TsOhv+9A3QlIrHYq4MPOAeYsOAK1kzWlccxao+
MTK+Hh1bXtRhH+LerYpVp3gDOU94wikO2RUwymFIN12iatw3rAugbmg02VdIY7gs
cc5/yY6o/73Gcu9ocMPaOMtODt0RczN/iS4Quonx8KRx416wztLciZtphkiPa3x1i
```

Task 7 – PHP Front-End Deployment with Templates

This task deploys a PHP web application and configures Nginx using templates.

Steps

1. Create required directories for files and templates.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ mkdir -p files templates
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ touch files/index.php
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ touch templates/nginx.conf.j2
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ls -R

.:
ansible.cfg  locals.tf  my-playbook.yaml  terraform.tfstate  variables.tf
files        main.tf    outputs.tf         terraform.tfstate.backup
hosts        modules   templates          terraform.tfvars

./files:
index.php

./modules:
subnet  webserver

./modules/subnet:
main.tf  outputs.tf  variables.tf

./modules/webserver:
main.tf  outputs.tf  variables.tf

./templates:
nginx.conf.j2
```

2. Create a PHP metadata web page.

```
lab_14 > Task > files > index.php
1  <?php
2  // Get hostname
3  $hostname = gethostname();
4
5  // Deployment date
6  $deployed_date = date("Y-m-d H:i:s");
7
8  // Metadata base URL
9  $metadata_base = "http://169.254.169.254/latest/";
10
11 // Function to get IMDSv2 token
12 function getImdsV2Token() {
```

3. Create an Nginx configuration template using Jinja2.

```
lab_14 > Task > templates > nginx.conf.j2
1  user nginx;
2  worker_processes auto;
3  error_log /var/log/nginx/error.log notice;
4  pid /run/nginx.pid;
5
6  events {
7      worker_connections 1024;
8  }
9
10 http {
11     log_format main '$remote_addr - $remote_user [$time local] "$request"'
```

4. Update the playbook to deploy PHP, copy files, and restart services.

```
lab_14 > Task > my-playbook.yaml
23  - name: Configure SSL certificates
26    tasks:
73
74  - name: Deploy Nginx website and configuration files
75    hosts: nginx
76    become: true
77    tasks:
78      - name: install php-fpm and php-curl
79        yum:
80          name:
81            - php-fpm
82            - php-curl
```

5. Run the playbook and verify the PHP application over HTTPS.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cd /workspaces/CC-HajraSarwar-022/lab_14/Task && ansible-playbook -i hosts my-playbook.yaml
TASK [install php-fpm and php-curl] *****
ok: [3.28.74.36]

TASK [Copy website files] *****
ok: [3.28.74.36]

TASK [Copy nginx.conf template] *****
changed: [3.28.74.36]

TASK [Restart nginx] *****
changed: [3.28.74.36]

TASK [Start and enable php-fpm] *****
changed: [3.28.74.36]

PLAY RECAP *****
3.28.74.36 : ok=19 changed=3 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

← → ↻ 🏠 🔒 Not secure https://3.28.74.36



Your connection is not private

Attackers might be trying to steal your information from 3.28.74.36 (for example, passwords, messages, or credit cards). [Learn more about this warning](#)

NET-ERR_CERT_AUTHORITY_INVALID



Turn on enhanced protection to get Chrome's highest level of security

Advanced

Back to safety

← → ↻ 🏠 🔒 Not secure https://3.28.74.36



Nginx Front End Web Server

Hostname: ip-10-0-10-145.me-central-1.compute.internal

Instance ID: i-015bf149f21462dc6

Private IP: 10.0.10.145

Public IP: 3.28.74.36

Public DNS: https://ec2-3-28-74-36.me-central-1.compute.amazonaws.com

Deployed: 2026-01-28 19:26:59

Status:  Active and Running

Managed By: Terraform + Ansible



This task installs Docker and Docker Compose on a fresh EC2 instance.

Steps

1. Destroy existing infrastructure using Terraform.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform destroy -auto-app
rove
Destroy complete! Resources: 7 destroyed.
```

2. Recreate a fresh EC2 instance.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform apply -auto-approve
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
```

```

@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform output
webserver_public_ips = [
    "40.172.234.49",
]

```

3. Update Ansible inventory for Docker servers.

```
lab_14 > Task > ≡ hosts
1  [docker_servers]
2  40.172.234.49
3
4  [docker_servers:vars]
5  ansible_ssh_private_key_file=~/.ssh/id_ed25519
6  ansible_user=ec2-user
```

4. Replace the playbook to install Docker and Docker Compose.

```
lab_14 > Task > ! my-playbook.yaml
1  ---
2  - name: Configure Docker
3    hosts: all
4    become: true
5    tasks:
6      - name: install docker and update cache
7        yum:
```

- Run the playbook and verify Docker installation via SSH.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i hosts m
y-playbook.yaml1
TASK [restart docker service] *****
changed: [40.172.234.49]

PLAY RECAP *****
40.172.234.49      : ok=8    changed=4    unreachable=0    failed=0    skipped=0    resc
```

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ssh ec2-user@40.172.234.49
-i ~/.ssh/id_ed25519
```

```
** The server may need to be upgraded. See https://openssh.com/pq.html
```

```

#_
~\####_ Amazon Linux 2023
~~~\#####\
~~~\###|
~~~\#/_
~~~V~' '->
~~~~~
~~~~~_+~_/_/
~~~~~/_/_/_/
~~~~~/_m/'

```

```
Last login: Wed Jan 28 19:37:09 2026 from 4.240.39.194
```

```
[ec2-user@ip-10-0-10-136 ~]$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

```
[ec2-user@ip-10-0-10-136 ~]$ exit
```

logout

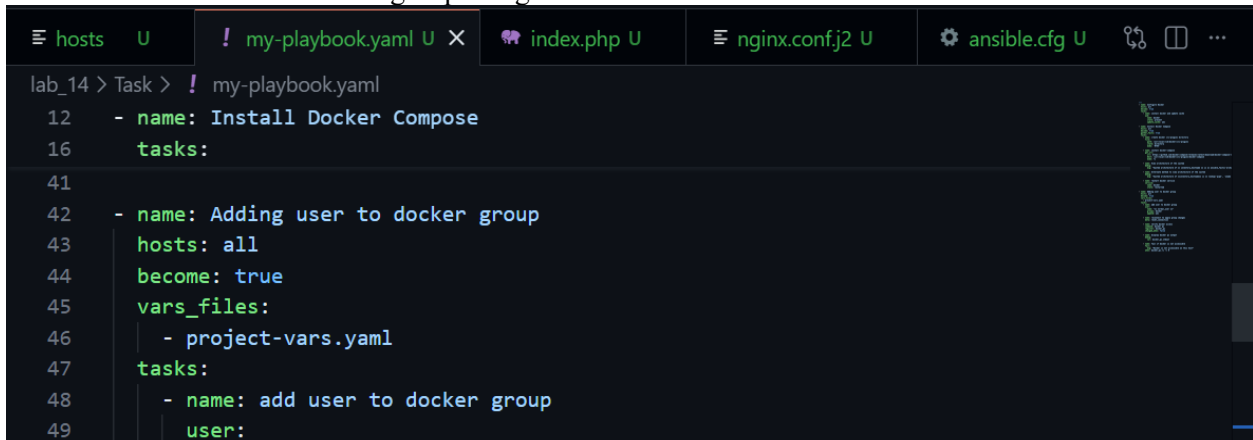
Connection to 40.172.234.49 closed.

Task 9 – Gitea Docker Stack via Ansible & Terraform Security Group Update

This task deploys a Gitea and PostgreSQL Docker stack and updates Terraform security groups.

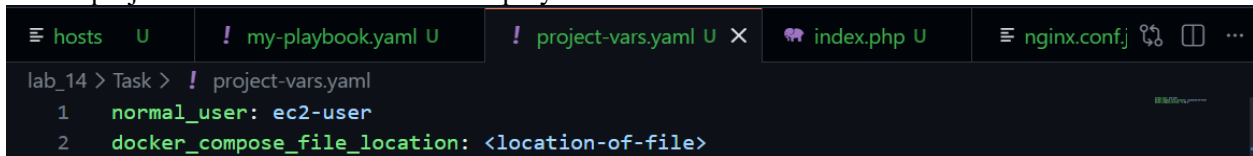
Steps

1. Add the EC2 user to the Docker group using Ansible.



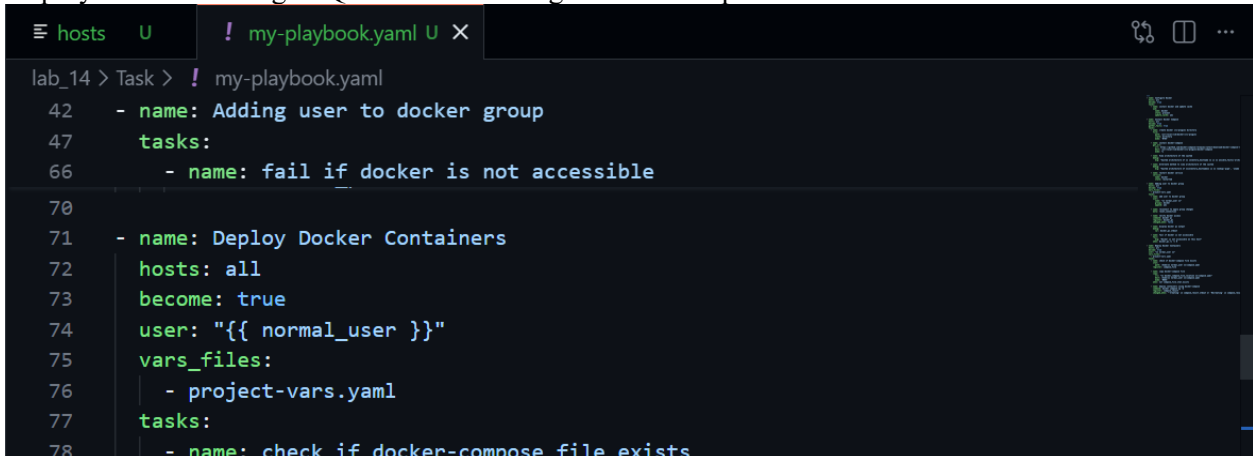
```
lab_14 > Task > ! my-playbook.yaml
12  - name: Install Docker Compose
16    tasks:
41
42  - name: Adding user to docker group
43    hosts: all
44    become: true
45    vars_files:
46      - project-vars.yaml
47    tasks:
48      - name: add user to docker group
49        user:
```

2. Create project variable files for Docker deployment.



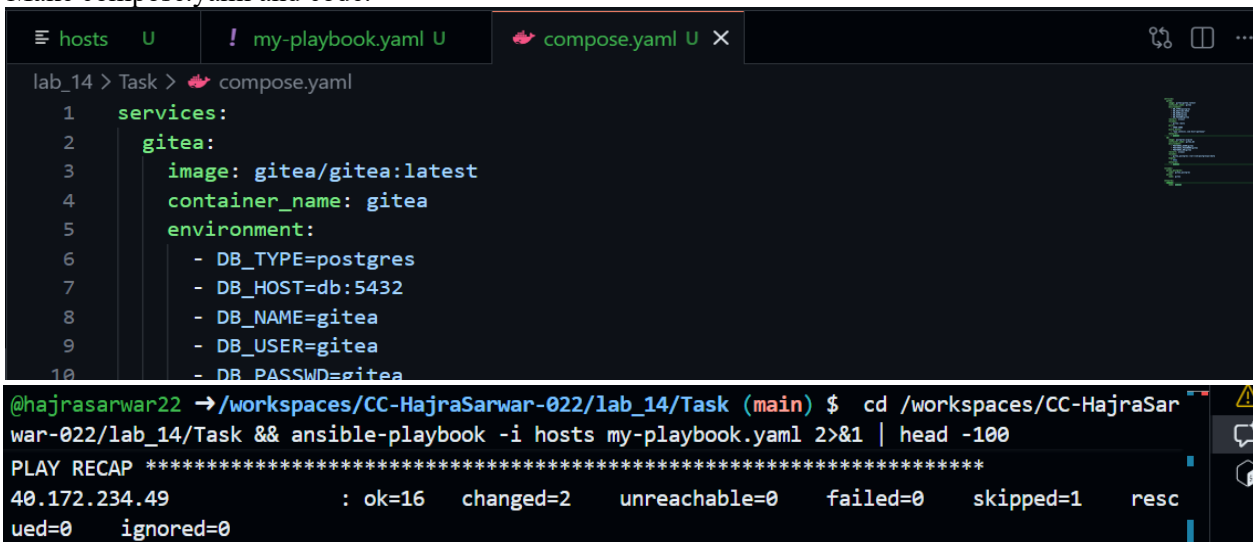
```
lab_14 > Task > ! project-vars.yaml
1  normal_user: ec2-user
2  docker_compose_file_location: <location-of-file>
```

3. Deploy Gitea and PostgreSQL containers using Docker Compose.



```
lab_14 > Task > ! my-playbook.yaml
42  - name: Adding user to docker group
47    tasks:
66      - name: fail if docker is not accessible
70
71  - name: Deploy Docker Containers
72    hosts: all
73    become: true
74    user: "{{ normal_user }}"
75    vars_files:
76      - project-vars.yaml
77    tasks:
78      - name: check if docker-compose file exists
```

4. Make compose.yaml and code.



```
lab_14 > Task > compose.yaml
1  services:
2    gitea:
3      image: gitea/gitea:latest
4      container_name: gitea
5      environment:
6        - DB_TYPE=postgres
7        - DB_HOST=db:5432
8        - DB_NAME=gitea
9        - DB_USER=gitea
10       - DB_PASSWORD=gitea

@hajrasarwar22 -> /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cd /workspaces/CC-HajraSarwar-022/lab_14/Task && ansible-playbook -i hosts my-playbook.yaml 2>&1 | head -100
PLAY RECAP *****
40.172.234.49      : ok=16   changed=2    unreachable=0    failed=0    skipped=1    resc
ued=0    ignored=0
```

5. Update Terraform security group to allow port 3000.


```
lab_14 > Task > modules > webserver > main.tf
1  resource "aws_security_group" "web_sg" {
18    ingress {
23    }
24    ingress {
25      from_port = 3000
26      to_port   = 3000
27      protocol  = "tcp"
28      cidr_blocks = ["0.0.0.0/0"]

```

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cd /workspaces/CC-HajraSarwar-022/lab_14/Task && terraform apply -auto-approve 2>&1 | tail -30
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

webserver_public_ips = [
  "40.172.234.49",

```

6. Apply Terraform changes and access Gitea via browser.

← → 🔍 ⚙️ ⚠️ Not secure 40.172.234.49:3000 ☆ 📄 🗑️ 🖨️ 🛠️ 🔍 ⌂

Initial Configuration

If you run Gitea inside Docker, please read the [documentation](#) before changing any settings.

Database Settings

Gitea requires MySQL, PostgreSQL, MSSQL, SQLite3 or TiDB (MySQL protocol).

Database Type *

Host *

Username *

Password *

Database Name *

SSL *

Schema

Leave blank for database default ("public").

Port number the Gitea web server will listen on.

Gitea Base URL *

Base address for HTTP(S) clone URLs and email notifications.

Log Path *

Log files will be written to this directory.

☐ Enable Update Checker

Checks for new version releases periodically by connecting to gitea.io.

Optional Settings

- ▶ Email Settings
- ▶ Server and Third-Party Service Settings
- ▶ Administrator Account Settings

These configuration options will be written into: 📄

Task 10 – Automating Ansible with Terraform (null_resource)

This task integrates Terraform and Ansible by using a Terraform `null_resource` with a `local-exec` provisioner. Terraform automatically triggers Ansible playbooks after EC2 creation to configure applications without manual intervention.

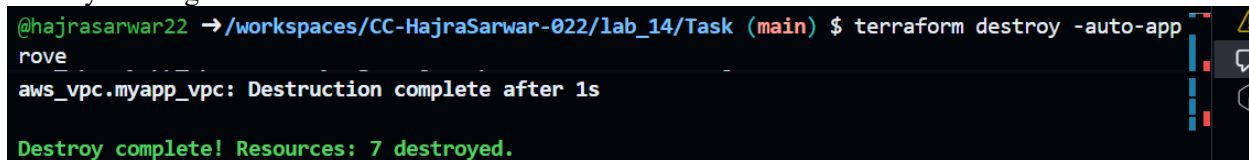
Steps

1. Update `main.tf` and add a `null_resource` that triggers Ansible using EC2 public IPs from Terraform outputs.



```
39 resource "null_resource" "configure_server" {
40   triggers = {
41     webserver_public_ips_for_ansible = join(",", [for i in module.myapp-webserver :
42   ]
43 }
44 depends_on = [module.myapp-webserver]
45
46 provisioner "local-exec" {
47   command = <<-EOT
48     ansible-playbook -i ${self.triggers.webserver_public_ips_for_ansible}, \
```

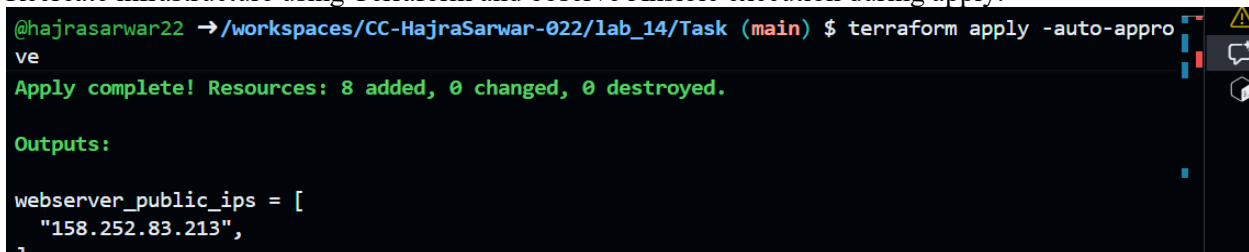
2. Configure the `local-exec` provisioner to execute `ansible-playbook` with private key and EC2 user.
3. Destroy existing infrastructure to ensure a clean run.



```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform destroy -auto-approve
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
```

4. Recreate infrastructure using Terraform and observe Ansible execution during apply.

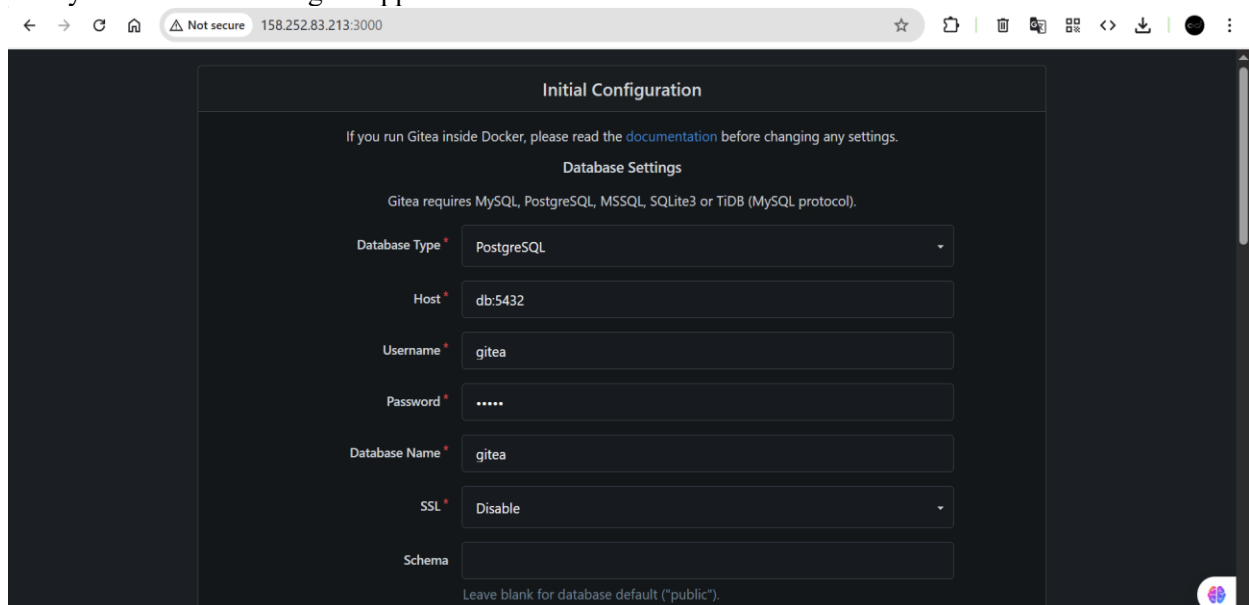


```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform apply -auto-approve
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

webserver_public_ips = [
  "158.252.83.213",
]
```

5. If Ansible fails due to EC2 readiness, add a `wait-for-SSH` task at the top of `my-playbook.yaml`.
6. Destroy infrastructure again and reapply Terraform.
7. Verify that the Gitea or Nginx application is reachable via browser after automation.



Initial Configuration

If you run Gitea inside Docker, please read the [documentation](#) before changing any settings.

Database Settings

Gitea requires MySQL, PostgreSQL, MSSQL, SQLite3 or TiDB (MySQL protocol).

Database Type * PostgreSQL

Host * db:5432

Username * gitea

Password *

Database Name * gitea

SSL * Disable

Schema

Leave blank for database default ("public").

Task 11 – Dynamic Inventory with aws_ec2 Plugin

This task configures Ansible to dynamically discover EC2 instances using the `aws_ec2` inventory plugin instead of maintaining static host files.

Steps

1. Update `ansible.cfg` to enable the `aws_ec2` inventory plugin and set required defaults.

```
lab_14 > Task > ⚙️ ansible.cfg
1  [defaults]
2  host_key_checking=False
3  interpreter_python = /usr/bin/python3
4  deprecation_warnings = False
5
6  enable_plugins = aws_ec2
7  private_key_file = ~/.ssh/id_ed25519
```

2. Create a new dynamic inventory file named `inventory_aws_ec2.yaml`.

```
• @hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ touch inventory_aws_ec2.yaml
1
• @hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ls -la inventory_aws_ec2.yaml
-rw-rw-rw- 1 vscode vscode 0 Jan 28 20:19 inventory_aws_ec2.yaml
```

3. Add initial plugin configuration and AWS region details.

```
lab_14 > Task > ! inventory_aws_ec2.yaml
1  ---
2  plugin: aws_ec2
3  regions:
4  - me-central-1
```

4. Update Terraform configuration to create both **dev** and **prod** EC2 instances.

```
lab_14 > Task > 🐜 main.tf
39  module "myapp-webserver-prod" {
48
49      # Loop count
50      count          = 1
51      # Use count.index to differentiate instances
52      instance_suffix = count.index
53  }
54
55  resource "null_resource" "configure_server" {
56      triggers = {
57          webserver_public_ips_for_aws = join(",", [for i in module.myapp-webserver :
```

5. Add Terraform outputs to display public IPs for both environments.

```
lab_14 > Task > 🐜 outputs.tf
1  output "webserver_public_ips" {
2      value = [for i in module.myapp-webserver : i.aws_instance.public_ip]
3  }
4
5  output "prod-webserver_public_ips" {
6      value = [for i in module.myapp-webserver-prod : i.aws_instance.public_ip]
7  }
```

6. Initialize and apply Terraform to build infrastructure.

```
@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform apply -auto-approve
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
• @hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform output
prod-webserver_public_ips = [
    "3.28.137.90",
]
webserver_public_ips = [
    "158.252.83.213",
]
```

7. Install `boto3` and `botocore` Python libraries required by the plugin.

```

@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ sudo apk add py3-boto3 py3-
-botocore && python3 -c "import boto3, botocore; print(f'boto3: {boto3.__version__}')"
(1/6) Installing py3-botocore (1.40.75-r0)
(2/6) Installing py3-botocore-pyc (1.40.75-r0)
(3/6) Installing py3-s3transfer (0.13.0-r0)
(4/6) Installing py3-s3transfer-pyc (0.13.0-r0)
(5/6) Installing py3-boto3 (1.40.75-r0)
(6/6) Installing py3-boto3-pyc (1.40.75-r0)
OK: 1157.9 MiB in 285 packages
boto3: 1.40.75

```

- Verify plugin functionality by generating the Ansible inventory graph.

```

@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-inventory -i invent
ory_aws_ec2.yaml --graph
@all:
|--@ungrouped:
|--@aws_ec2:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com

```

Task 12 – Filtering EC2 Instances by Tags & Instance Type

This task enhances the dynamic inventory by grouping EC2 instances based on AWS tags and instance types, allowing targeted Ansible execution.

Steps

- Update `inventory_aws_ec2.yaml` to enable grouping based on instance tags.

```

lab_14 > Task > ! inventory_aws_ec2.yaml
1  ---
2  plugin: aws_ec2
3  regions:
4  - me-central-1
5
6  keyed_groups:
7  - key: tags
8    prefix: tag
9    separator: "_"

```

- Generate the Ansible inventory graph to verify tag-based grouping.

```

@hajrasarwar22 →/workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-inventory -i invent
ory_aws_ec2.yaml --graph
@all:
|--@ungrouped:
|--@aws_ec2:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
|--@tag_Name_prod_ec2_instance_0:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
|--@tag_Name_dev_ec2_instance_0:
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com

```

- Extend the inventory configuration to group instances by instance type.

```

lab_14 > Task > ! inventory_aws_ec2.yaml
3  regions:
5
6  keyed_groups:
7  - key: tags
8    prefix: tag
9    separator: "_"
10
11  - key: instance_type
12    prefix: instance_type
13    separator: "_"

```

4. Verify updated inventory graph showing both tag and instance-type groups.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-inventory -i inventory_aws_ec2.yaml --graph
@all:
|--@ungrouped:
|--@aws_ec2:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
|--@tag_Name_prod_ec2_instance_0:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
|--@instance_type_t3_nano:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
|--@tag_Name_dev_ec2_instance_0:
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
|--@instance_type_t3_micro:
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
```

5. Update my-playbook.yaml to support execution on all hosts.

```
lab_14 > Task > ! my-playbook.yaml
1  ---
2  - name: Configure nginx web server
3    hosts: all
4    become: true
5    tasks:
6      - name: install nginx and update cache
7        yum:
8          name: nginx
9          state: present
10         update_cache: yes
```

6. Run the playbook on all EC2 instances.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i inventory_aws_ec2.yaml my-playbook.yaml
TASK [Restart nginx] *****
changed: [ec2-3-28-137-90.me-central-1.compute.amazonaws.com]
changed: [ec2-158-252-83-213.me-central-1.compute.amazonaws.com]

TASK [Start and enable php-fpm] *****
changed: [ec2-3-28-137-90.me-central-1.compute.amazonaws.com]
changed: [ec2-158-252-83-213.me-central-1.compute.amazonaws.com]

PLAY RECAP *****
ec2-158-252-83-213.me-central-1.compute.amazonaws.com : ok=18  changed=10  unreachable=0  failed=0
ec2-3-28-137-90.me-central-1.compute.amazonaws.com : ok=18  changed=10  unreachable=0  failed=0
```

7. Execute the playbook selectively for dev, prod, and specific instance types.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i inventory_aws_ec2.yaml -l tag_Name_dev_* my-playbook.yaml
PLAY RECAP *****
ec2-158-252-83-213.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  failed=0

@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i inventory_aws_ec2.yaml -l tag_Name_prod_* my-playbook.yaml
PLAY RECAP *****
ec2-3-28-137-90.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  failed=0

@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i inventory_aws_ec2.yaml -l instance_type_t3_micro my-playbook.yaml
PLAY RECAP *****
ec2-158-252-83-213.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  failed=0

@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -i inventory_aws_ec2.yaml -l instance_type_t3_nano my-playbook.yaml
PLAY RECAP *****
ec2-3-28-137-90.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  failed=0
```

- Set the dynamic inventory as default in `ansible.cfg`.

```
lab_14 > Task > ansible.cfg
1 [defaults]
2 host_key_checking=False
3 interpreter_python = /usr/bin/python3
4 deprecation_warnings = False
5 remote_user = ec2-user
6 private_key_file = ~/.ssh/id_ed25519
7 inventory = ./inventory_aws_ec2.yaml
```

- Run Ansible without specifying the inventory explicitly.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ ansible-playbook -l instance_type_t3_nano my-playbook.yaml
PLAY RECAP *****
ec2-3-28-137-90.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  failed=0
d=0  skipped=0  rescued=0  ignored=0
```

Task 13 – Ansible Roles: nginx, ssl, webapp

This task refactors the Ansible configuration into reusable roles, improving modularity, readability, and scalability.

Steps

- Update Terraform to deploy a single dev EC2 instance.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cd /workspaces/CC-HajraSarwar-022/lab_14/Task && cat main.tf
my_ip = local.my_ip
vpc_id = aws_vpc.myapp_vpc.id
subnet_id = module.myapp-subnet.subnet.id

# Loop count
count = 1
# Use count.index to differentiate instances
instance_suffix = count.index
```

- Create a structured Ansible directory with inventory, roles, and configuration files.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ mkdir -p ansible
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cd ansible
@hajrasarwar22 → .../CC-HajraSarwar-022/lab_14/Task/ansible (main) $ mkdir inventory roles
@hajrasarwar22 → .../CC-HajraSarwar-022/lab_14/Task/ansible (main) $ touch ansible.cfg my-playbook.yaml
@hajrasarwar22 → .../CC-HajraSarwar-022/lab_14/Task/ansible (main) $ ls -R
.:
ansible.cfg  inventory  my-playbook.yaml  roles

./inventory:

./roles:
```

- Configure project-level `ansible.cfg` and static inventory.

```
lab_14 > Task > ansible > ansible.cfg
1 [defaults]
2 host_key_checking=False
3 interpreter_python = /usr/bin/python3

lab_14 > Task > ansible > inventory > hosts
1 [nginx]
2 158.252.83.213
3
4 [nginx:vars]
5 ansible_ssh_private_key_file=~/.ssh/id_ed25519
6 ansible_user=ec2-user
```


4. Create roles for **nginx**, **ssl**, and **webapp** using **ansible-galaxy**.

```
@hajrasarwar22 →.../lab_14/Task/ansible/roles (main) $ ansible-galaxy role init nginx
- Role nginx was created successfully
@hajrasarwar22 →.../lab_14/Task/ansible/roles (main) $ ansible-galaxy role init ssl
- Role ssl was created successfully
@hajrasarwar22 →.../lab_14/Task/ansible/roles (main) $ ansible-galaxy role init webapp
- Role webapp was created successfully
@hajrasarwar22 →.../lab_14/Task/ansible/roles (main) $ cd ..
@hajrasarwar22 →.../CC-HajraSarwar-022/lab_14/Task/ansible (main) $ tree
```

```
.
├── ansible.cfg
├── inventory
│   └── hosts
├── my-playbook.yaml
└── roles
    ├── nginx
    │   ├── README.md
    │   ├── defaults
    │   │   └── main.yml
    │   ├── files
    │   ├── handlers
    │   │   └── main.yml
    │   ├── meta
    │   │   └── main.yml
    │   ├── tasks
    │   │   └── main.yml
    │   ├── templates
    │   ├── tests
    │   │   ├── inventory
    │   │   └── test.yml
    │   └── vars
    │       └── main.yml
    ├── ssl
    │   ├── README.md
    │   ├── defaults
    │   │   └── main.yml
    │   ├── files
    │   ├── handlers
    │   │   └── main.yml
    │   ├── meta
    │   │   └── main.yml
    │   ├── tasks
    │   │   └── main.yml
    │   ├── templates
    │   ├── tests
    │   │   ├── inventory
    │   │   └── test.yml
    │   └── vars
    │       └── main.yml
    └── webapp
        ├── README.md
        ├── defaults
        │   └── main.yml
        ├── files
        ├── handlers
        │   └── main.yml
        ├── meta
        │   └── main.yml
        ├── tasks
        │   └── main.yml
        ├── templates
        ├── tests
        │   ├── inventory
        │   └── test.yml
        └── vars
            └── main.yml
```

29 directories, 27 files

5. Implement handlers and tasks for the nginx role.

```
! main.yml U X
lab_14 > Task > ansible > roles > nginx > handlers > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # handlers file for nginx
4  - name: Restart nginx
5    service:
6      name: nginx
7      state: restarted

! main.yml .../handlers U ! main.yml .../tasks U X
lab_14 > Task > ansible > roles > nginx > tasks > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # tasks file for nginx
4  - name: Install nginx
5    yum:
6      name: nginx
7      state: present
8      update_cache: yes
9      notify: Restart nginx
10
11 - name: Install openssl
12   yum:
13     name: openssl
```

6. Test nginx role independently using a role-based playbook.

```
lab_14 > Task > ansible > ! my-playbook.yaml
1  ---
2  - name: Deploy NGINX Web Stack with SSL and PHP
3    hosts: nginx
4    become: true
5    roles:
6      - nginx

@hajrasarwar22 → .../CC-HajraSarwar-022/lab_14/Task/ansible (main) $ ansible-playbook -i inventory/hosts my-playbook.yaml
TASK [nginx : Install openssl] *****
ok: [158.252.83.213]

TASK [nginx : Start and enable nginx] *****
ok: [158.252.83.213]

PLAY RECAP *****
158.252.83.213 : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Initial Configuration

If you run Gitea inside Docker, please read the [documentation](#) before changing any settings.

Database Settings

Gitea requires MySQL, PostgreSQL, MSSQL, SQLite3 or TiDB (MySQL protocol).

Database Type *

Host *

Username *

Password *

Database Name *

SSL *

Schema

Leave blank for database default ("public").

7. Implement SSL role to generate self-signed certificates dynamically.

```
lab_14 > Task > ansible > roles > ssl > defaults > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # defaults file for ssl
4  imdsv2_token_ttl: "3600"
5  ssl_days_valid: 365
```

```
lab_14 > Task > ansible > roles > ssl > tasks > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # tasks file for ssl
4  - name: Create SSL private directory
5    file:
6      path: /etc/ssl/private
7      state: directory
8      mode: '0700'
```

8. Implement webapp role for PHP application deployment and Nginx templating.

```
lab_14 > Task > ansible > roles > webapp > defaults > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # defaults file for webapp
4  nginx_user: nginx
5  nginx_worker_processes: auto
6  nginx_worker_connections: 1024
7  nginx_error_log_level: notice
```

```
lab_14 > Task > ansible > roles > webapp > files > ! index.php
1  <?php
2  // Get hostname
3  $hostname = gethostname();
4
5  // Deployment date
6  $deployed_date = date("Y-m-d H:i:s");
7
8  // Metadata base URL
9  $metadata_base = "http://169.254.169.254/latest/";
```

```
lab_14 > Task > ansible > roles > webapp > handlers > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # handlers file for webapp
4  - name: Restart nginx
5    service:
6      name: nginx
7      state: restarted
8
9  - name: Restart php-fpm
```

```
lab_14 > Task > ansible > roles > webapp > templates > ! nginx.confj2
1  user {{ nginx_user }};
2  worker_processes {{ nginx_worker_processes }};
3  error_log /var/log/nginx/error.log {{ nginx_error_log_level }};
4  pid /run/nginx.pid;
5
6  events {
7    worker_connections {{ nginx_worker_connections }};
```

```
lab_14 > Task > ansible > roles > webapp > tasks > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # tasks file for webapp
4  - name: Install PHP packages
5    yum:
6      name:
7        - php-fpm
8        - php-curl
9      state: present
```

9. Create a final playbook combining all roles.

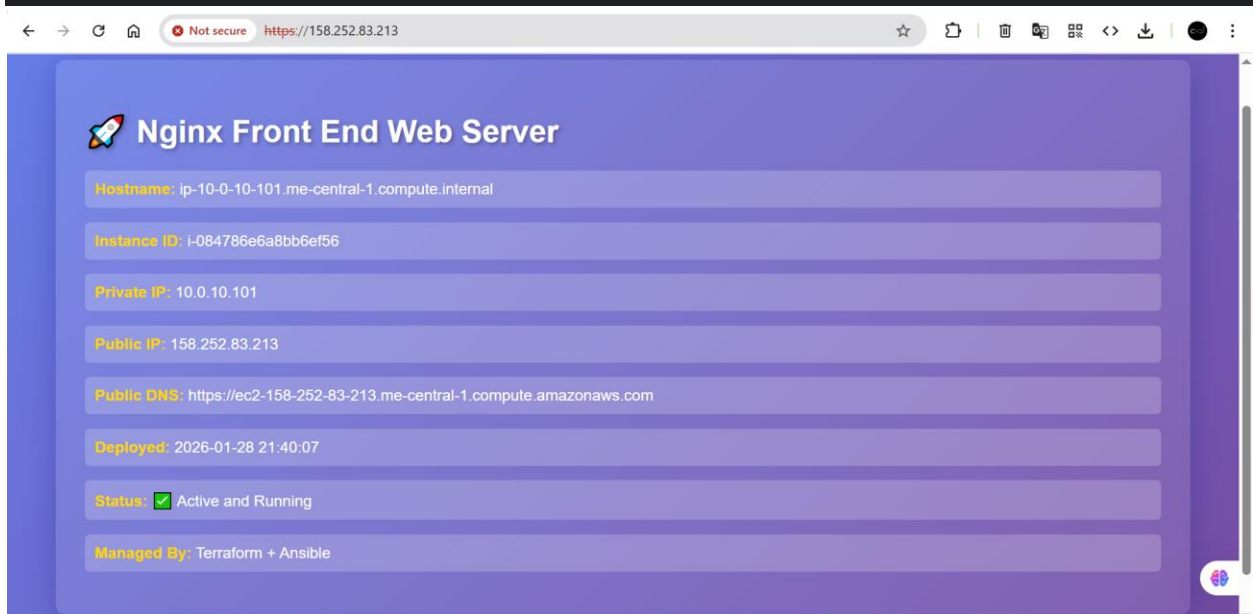
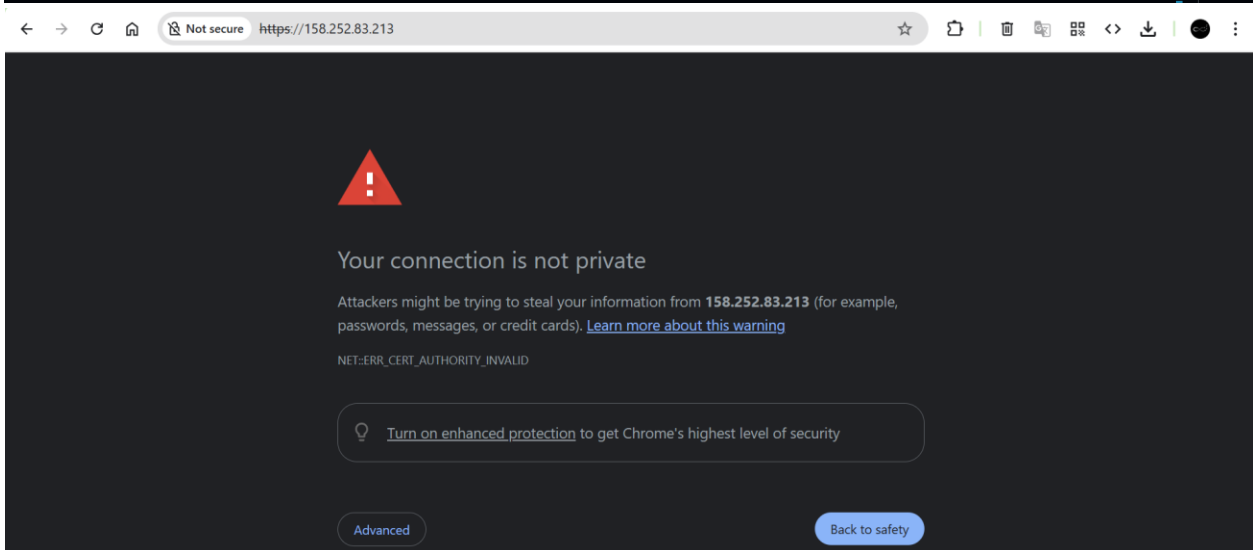
```
! my-playbook.yaml U x ! main.yml .../ssl/defaults U ! main.yml .../ssl/tasks U ! main.yml .../webapp U ...
lab_14 > Task > ansible > ! my-playbook.yaml
1 ---
2 - name: Deploy NGINX Web Stack with SSL and PHP
3   hosts: nginx
4   become: true
5   roles:
6     - nginx
7     - ssl
8     - webapp
```

10. Run the complete role-based playbook and verify HTTPS PHP application.

```
@hajrasarwar22 → .../CC-HajraSarwar-022/lab_14/Task/ansible (main) $ ansible-playbook -i inventory/hosts my-playbook.yaml
TASK [webapp : Start and enable php-fpm] *****
ok: [158.252.83.213]

RUNNING HANDLER [webapp : Restart nginx] *****
changed: [158.252.83.213]

PLAY RECAP *****
158.252.83.213 : ok=15 changed=3 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```



Cleanup – Destroy Resources & Verify State

This task ensures that all AWS resources created during the lab are properly destroyed and no infrastructure remains active.

Steps

1. Destroy all Terraform-managed resources using `terraform destroy`.

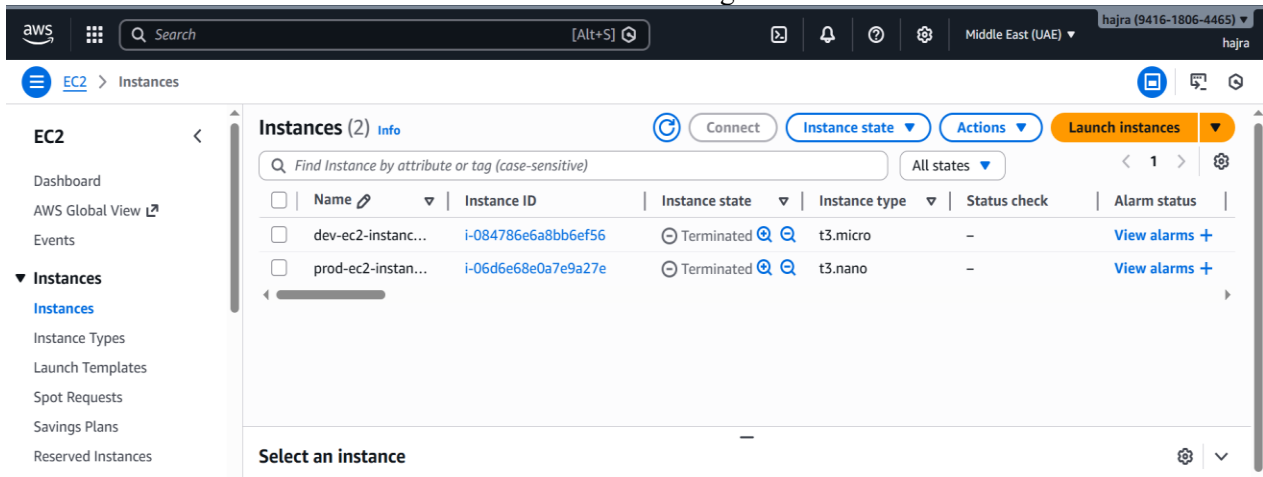
```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ terraform destroy -auto-approve
module.myapp-webserver-prod[0].aws_security_group.web_sg: Destroying... [id=sg-05bd3bea37989e92b]
module.myapp-webserver-prod[0].aws_key_pair.ssh-key: Destruction complete after 0s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destruction complete after 1s
module.myapp-webserver-prod[0].aws_security_group.web_sg: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-0c35f9c605b05e4c8]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 11 destroyed.
```

2. Verify Terraform state file shows no active resources.

```
@hajrasarwar22 → /workspaces/CC-HajraSarwar-022/lab_14/Task (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.10.0",
  "serial": 76,
  "lineage": "486b1b4b-1187-bcdc-c3f5-58d8512b2ade",
  "outputs": {},
  "resources": [],
  "check_results": null
}
```

3. Confirm via AWS Console that no EC2 instances are running.



The screenshot shows the AWS Management Console interface for the EC2 service. The left sidebar contains navigation links for EC2, Dashboard, AWS Global View, Events, and a list of instance types (Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances). The main content area is titled 'Instances (2)' and shows a table of two EC2 instances. Both instances are in a 'Terminated' state. The table columns are Name, Instance ID, Instance state, Instance type, Status check, and Alarm status.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status
dev-ec2-instanc...	i-084786e6a8bb6ef56	Terminated	t3.micro	-	View alarms +
prod-ec2-instan...	i-06d6e68e0a7e9a27e	Terminated	t3.nano	-	View alarms +